

Light You Can Address

One data wire, every LED its own color — a strict-timing 800 kHz protocol with the level-shift and power discipline that decides whether the color is solid or glitchy.

ESP32-006 · ~40 MIN · INTERMEDIATE

1. The one idea

A WS2812B strip is a chain of LEDs on **one data wire**, where each LED reads its own color off the front of the stream and passes the rest down the line. The colors are encoded in pulse *width* at 800 kHz — strict timing the library handles, wrapped in the power and logic-level discipline you have to handle.

2. Why it matters

Addressable LEDs are everywhere — signs, ambient light, status bars. But the protocol is timing-critical and 5V-native, so this build teaches two hard skills at once: precise serial timing, and the power integrity (level-shifting, bulk capacitance) that separates rock-solid color from a flickering, first-pixel-wrong mess.

3. Parts & wiring

- ESP32 dev board
- WS2812B RGB LED stick (8 px)
- 1000 μ F electrolytic capacitor (across the 5V rail at the strip)
- 330–470 Ω resistor in series with the data line
- A 5V supply for the LEDs (bench PSU or a 5V rail), **separate from the ESP32**
- A 3.3V→5V level shifter for the data line (see the guard)

Node	Connect	Why
ESP32 GPIO5	→330 Ω → level shifter → strip DIN	series R tames ringing; shifter lifts logic
Strip 5V	5V rail, with 1000 μ F right at the input	LEDs draw current in bursts
Strip GND	GND — shared with the ESP32	one common ground

The two rules that make color solid. **Level-shift the data line:** a WS2812B wants a logic-high near $0.7 \times 5V \approx 3.5V$, but the ESP32 only drives 3.3V — marginal, and the first pixel often reads garbage. A 74AHCT125 buffer lifts 3.3V → a clean 5V. (That shifter isn't on the bench yet — it's on your Shopping list. Until it arrives you can sometimes run the strip at ~4.3V to lower the threshold, or sacrifice the first pixel as a level converter, but the buffer is the real fix.) **Bulk cap:** 1000μF across the 5V rail at the strip soaks the current spikes that otherwise sag the rail and flicker the colors. Power LEDs from their own 5V; share only ground.

4. Predict (do this before uploading)

- Eight LEDs at full white — roughly how much current? (each channel ~20 mA.)
- Drive the data line straight from 3.3V with no shifter — solid color, or flicker?
- When you flash the whole strip bright white at once, what happens to the 5V rail without the bulk cap?

5. Build it — name a pixel, give it a color

```
#include <Adafruit_NeoPixel.h>

#define DATA_PIN 5
#define N 8
Adafruit_NeoPixel strip(N, DATA_PIN, NEO_GRB + NEO_KHZ800);

void setup() {
  strip.begin();
  strip.setBrightness(60);           // current-limit: full white x8 is a lot
  strip.show();
}

void loop() {
  for (int i = 0; i < N; i++) {
    strip.clear();
    strip.setPixelColor(i, strip.Color(0, 120, 255)); // one pixel, addressed
    strip.show();
    delay(80);
  }
}
```

The new idea is the *address again*, in light: `setPixelColor(i, ...)` names one LED in the chain, and `show()` clocks the whole stream out with the exact timing the parts need. The library owns the nanoseconds; you own the volts and the amps.

6. Test against your prediction

A single blue pixel should run cleanly down the strip. If the first pixel glitches or the colors shimmer, that's the missing level shift and bulk cap talking — exactly the \$4 prediction. Add them and the color snaps solid.

7. What goes wrong (pre-loaded debugging)

- **First pixel is the wrong color / flickers** → 3.3V data is below the strip's logic-high threshold. Level-shift it.
- **Whole strip shimmers on bright changes** → the 5V rail is sagging under current bursts. Add the 1000µF right at the strip.
- **ESP32 browns out** → you powered the LEDs from the board. Give them a separate 5V rail; share only ground.
- **Colors are swapped (red shows green)** → wrong color order; use `NEO_GRB` (most WS2812B) vs `NEO_RGB`.
- **Random glitches on a long run** → ringing on the data line; keep it short and keep the 330Ω series resistor.

8. Checkpoint

You drove a strict-timing, one-wire LED protocol with the level-shift and power integrity it demands — and got solid, addressable color. The timing was the library's job; the volts, amps, and a clean logic edge were yours, and those are the skills that carry to every fast peripheral on the bench.

9. Push further

- Make it sound-reactive with the INMP441 mic (Course 9) — audio in, light out.
- Step up to the 64×64 matrix — far more current; plan the 5V rail and the cap before you plug it in.
- Add a gamma curve so brightness ramps look linear to the eye.